

Diagrama de Clases UML — Sistema de Biblioteca

Programación Orientada a Objetos

Boris Alberto Salle

Elizabeth Ramirez Agudelo

Marisol Ramirez Agudelo

Grupo 6

IU DIGITAL

Medellín

30 de Agosto 2026

1. Introducción

En el desarrollo de software, el Lenguaje Unificado de Modelado (UML) es una herramienta clave porque nos ayuda a planificar y estructurar un sistema orientado a objetos antes de ponernos a escribir código. Dentro de las distintas vistas que ofrece UML, el diagrama de clases funciona como el mapa principal de la arquitectura: nos permite identificar los objetos del mundo real, definir qué información van a guardar, qué acciones van a realizar y cómo se van a conectar entre sí. Trabajar con este tipo de diagramas desde el principio evita desorden en el código y facilita que el proyecto pueda crecer a futuro sin volverse inmanejable.

Para esta actividad colaborativa, abordamos el diseño de un Sistema de Gestión de Biblioteca. El objetivo es resolver los requerimientos típicos de este dominio (como el control del catálogo, el registro de autores, los préstamos a usuarios y el cálculo de multas), aplicando en la práctica los pilares de la Programación Orientada a Objetos: abstracción, encapsulamiento, herencia y polimorfismo. Además, estructuramos las clases cuidando que tengan una responsabilidad clara (alta cohesión) y que no dependan exageradamente unas de otras (bajo acoplamiento), guiándonos con los principios de diseño SOLID.

2. Objetivos

2.1 Objetivo General

Crear el diagrama de clases en UML para un sistema de biblioteca, aplicando los conceptos de abstracción, encapsulamiento, herencia y polimorfismo. La idea es lograr un diseño bien estructurado, con componentes independientes y fáciles de mantener, siguiendo las mejores prácticas de diseño orientado a objetos y los principios SOLID.

2.2 Objetivos Específicos

- **Definir las entidades principales del sistema:** Identificar clases clave como *Libro*, *Autor*, *Usuario* y *Préstamo*, configurando sus datos protegidos (atributos privados) con sus respectivos métodos para manipularlos correctamente.
- **Modelar las conexiones entre clases:** Determinar cómo interactúan los objetos entre sí mediante el uso de asociaciones, agregaciones, composiciones y herencia, buscando que el sistema no quede acoplado en exceso.
- **Aplicar polimorfismo para manejar distintos tipos de libros:** Implementar comportamientos específicos que permitan diferenciar el tratamiento de un *Libro Físico* frente a un *Libro Digital* (por ejemplo, en las reglas y formas de préstamo).
- **Incorporar principios SOLID en la arquitectura:** Diseñar las clases de modo que cada una tenga una única función bien clara (SRP) y que el sistema sea fácil de extender con nuevas funcionalidades sin tener que modificar el código ya existente (OCP).
- **Trabajar en equipo con herramientas de desarrollo:** Organizar el modelado del proyecto utilizando plataformas de diagramación colaborativa en la nube y repositorios en GitHub para gestionar los avances y la documentación.

3. Desarrollo de la Actividad

3.1 Descripción e Identificación de Clases

A partir del análisis de requerimientos del dominio del sistema de biblioteca, se formalizaron las clases principales y secundarias, definiendo sus responsabilidades específicas y encapsulamiento:

Clase	Atributos Clave	Métodos / Responsabilidad
Libro (Abstracta)	- isbn: String - titulo: String - anioPublicacion: int - disponible: boolean	+ obtenerInformacion(): String (Abstracto) + estaDisponible(): boolean + cambiarEstadoDisponibilidad (estado: boolean)
LibroFisico	- ubicacionEstante: String - estadoConservacion: String	+ obtenerInformacion(): String + calcularDiasPrestamo(): int
LibroDigital	- formatoArchivo: String - tamanoMB: double - urlDescarga: String	+ obtenerInformacion(): String + calcularDiasPrestamo(): int + descargar(): void
Autor	- idAutor: String - nombre: String - nacionalidad: String	+ agregarLibroEscrito(libro: Libro): void + obtenerObraCompleta(): List<Libro>
Prestamo	- idPrestamo: String - fechaPrestamo: Date - fechaDevolucionEsperada: Date - fechaDevolucionReal: Date	+ registrarDevolucion(): void + estaVencido(): boolean + calcularMulta(): double
DetallePrestamo	- diasConcedidos: int - tarifaDiariaMulta: double	+ calcularSubtotal(): double
Usuario	- idUsuario: String - nombre: String - email: String	+ puedeSolicitarPrestamo(): boolean

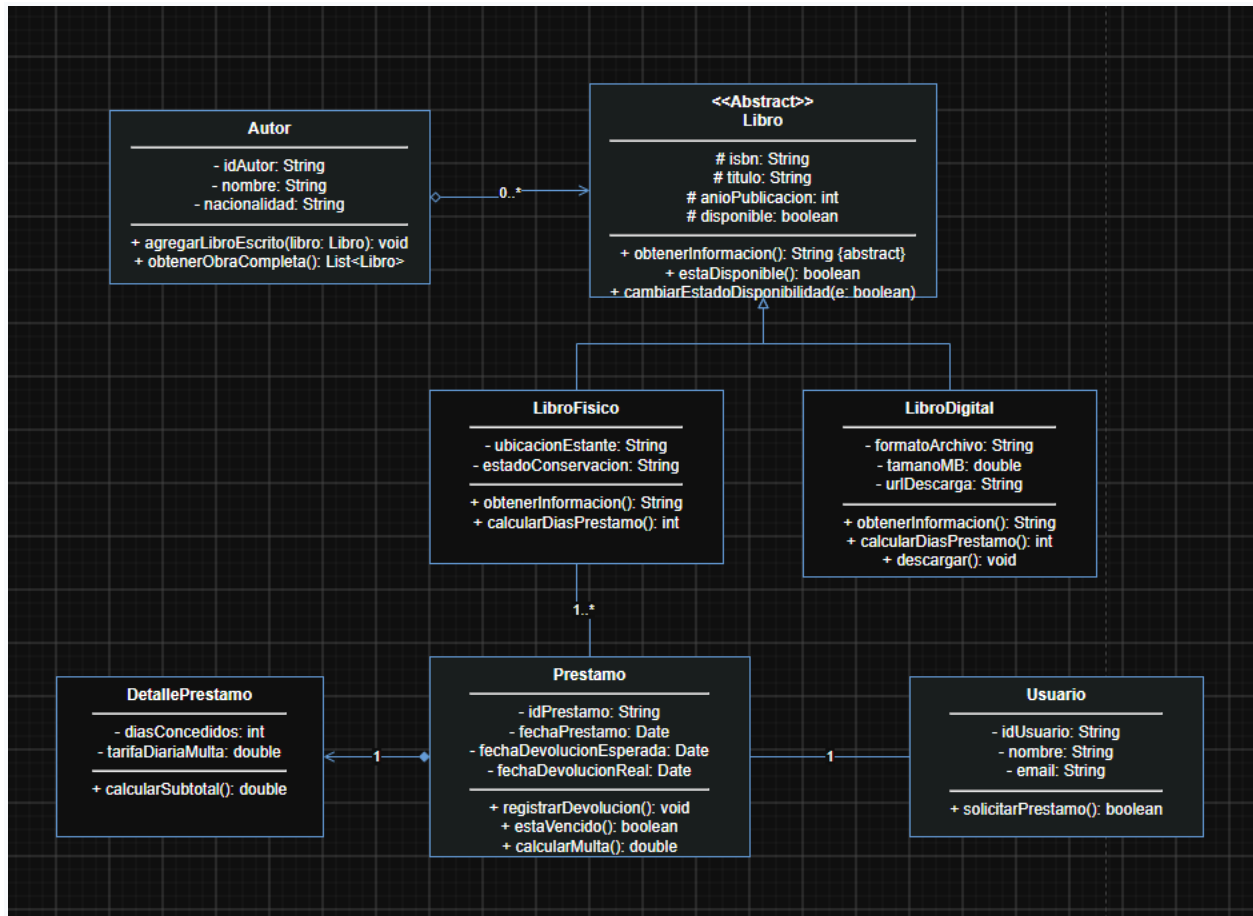
3.2 Justificación de las Relaciones UML y Multiplicidades

- **Agregación (Autor ◇--* Libro):** Un Autor posee una colección de libros escritos. El ciclo de vida del Libro es independiente del Autor; si el autor se elimina, las obras permanecen en el catálogo. Multiplicidad: 1 Autor a 0..* Libros.
- **Composición (Prestamo ◆--1 DetallePrestamo):** Un Prestamo está compuesto directamente por su DetallePrestamo. Si el registro de préstamo se anula o elimina, el detalle pierde su existencia operativa. Multiplicidad: 1 a 1.
- **Herencia (LibroFisico / LibroDigital --|> Libro):** La clase abstracta Libro encapsula los atributos comunes. Las subclases especializan propiedades específicas (ubicación en estante o tamaño de archivo MB).
- **Asociación Simple (Prestamo ---- Usuario / Prestamo ---- Libro):** Relación operativa de préstamo. Multiplicidad: Un Usuario puede tener 0..* préstamos; cada Préstamo está asociado a 1 Usuario y 1 Libro.

3.3 Aplicación de Cohesión, Acoplamiento y Principios SOLID

- **Alta Cohesión:** Cada clase se enfoca en un dominio exclusivo (Libro gestiona catálogo, Prestamo maneja fechas y reglas, GestionPrestamos ejecuta la lógica transaccional).
- **Bajo Acoplamiento:** Las relaciones de dependencia interactúan con abstracciones en lugar de acoplarse directamente a clases concretas.
- **Single Responsibility Principle (SRP):** Separación clara entre modelos de datos y servicios de lógica de negocio.
- **Open/Closed Principle (OCP):** El sistema es extensible a nuevos tipos de publicaciones sin modificar la clase base ni la lógica de préstamos existente.
- **Liskov Substitution Principle (LSP):** Las instancias de LibroFisico y LibroDigital sustituyen transparentemente cualquier uso de la abstracción Libro.

3.4 Diagrama de Clases UML del Sistema



4. Enlaces Verificados

Recurso Digital	Enlace de Acceso Verificado	Estado Permisos
Diagrama UML Interactivo (Draw.io)	https://drive.google.com/file/d/1uL8GQb_GQtuFp0jx_lccN7740LoS0R33/view?usp=drive_link	Público / Solo Lectura
Repositorio del Proyecto (GitHub)	https://github.com/elizabethramirez-a-eng/POO_Elizabeth_Marisol	Público (README & PlantUML)
Video Explicativo (Exposición)	https://www.youtube.com/watch?v=IFl2X9ppey4	No listado / Acceso público

5. Conclusiones

¿Qué se aprendió?

Al desarrollar el modelado de este Sistema de Biblioteca, pudimos poner en práctica cómo la abstracción y el encapsulamiento ayudan a proteger los datos de los objetos usando la visibilidad adecuada (atributos privados y protegidos) y controlando su acceso con los métodos correspondientes. Además, al crear la clase abstracta Libro y conectar sus subclases LibroFisico y LibroDigital, vimos el verdadero valor de la herencia y el polimorfismo: nos permite manejar diferentes tipos de libros de forma organizada y uniforme sin necesidad de enredar o duplicar la lógica de los préstamos.

¿En qué aporta a su desempeño profesional?

Para nuestro proceso como futuras tecnólogas en desarrollo de software, aprender a diagramar en UML antes de escribir código es una habilidad clave. Nos permite planificar bien la arquitectura del sistema, evitar errores desde el principio y ahorrar tiempo en correcciones cuando ya se está programando. También nos ayuda a mejorar la documentación técnica y la comunicación, lo cual es fundamental para trabajar de forma ordenada en equipos de desarrollo real.